

Comp Photo hw 1

Jenny Quan (jennyq)

July 2026

1 Developing Raw Images

1.1 Implement a basic image processing pipeline

RAW image conversion

The values reported by `dcraw -4 -d -v -w -T <>` are as follows:

```
<black> = 150
<white> = 4095
<r_scale> = 2.394531
<g_scale> = 1.000000
<b_scale> = 1.597656
```

Python Initials

The image has 16 bits per pixel, as well as a width of 6016 and a height of 4016.

Linearization

Nothing to include in report for this sub-task

Identifying the correct Bayer Pattern

The Bayer pattern I identified was `rggb`

White Balancing

I prefer the gray-world white balancing over the other two. See Figure 1.

Demosaicing

Nothing to include in report for this sub-task

Color space correction

Nothing to include in report for this sub-task

Brightness adjustment and gamma encoding



Figure 1: Left to right: White-world balancing, Gray-world balancing, White balancing with camera presets

After experimenting with some percentages, I preferred a mean value of 0.12.

Compression

At `quality=95`, I could not tell the difference between the two files. The compression ratio was $\frac{7.0}{39.1} \approx 0.18$.

By changing the quality setting, I found that I could lower the compression ratio to `quality=45` before I could distinguish the two images. The compression ratio here was $\frac{1.2}{39.1} \approx 0.03$.

1.2 Perform manual white balancing



Figure 2: Manual white balancing with different patches selected. Left: cloud, Center: white handrail, Right: wean building

Sampling the white handrail and sampling the wean building both worked well. The cloud sample did not work well, perhaps because the cloud had some blue hue, so calibrating to the cloud caused the rest of the image to appear yellow. The handrail patch selection feels like the most accurate color reproduction. Surprisingly, I prefer the Wean Hall patch selection to be most pleasing to the eye (even though Wean has more of a tan hue), perhaps because the overall result is more "balanced" in terms of color distribution.

Deliverables: .PNG files showing the results of manual white balancing for different patch selection

1.3 Learn to use `dcraw`

I used `dcraw -w -q 0 -o 1`.

We now have three versions of the developed image. I think the main differences between the three images is the color balance and the overall contrast. The image created by my implementation feels more "flat", while the shadows are harsher/deeper in the images produced by dcrow and the camera. The tree in the camera-produced image also has warmer highlights, even though the tree's shadows are not significantly more yellow compared to the other two images.

I think I prefer the image produced by the camera pipeline. I feel like the shadows in this image are the best at conveying a sense of depth.



Figure 3: Left to right: my implementation, dcrow, camera

Deliverables: .PNG file produced by `dcraw`

2 Camera Obscura

2.1 Build the pinhole camera

The screen size is 7in wide and 10 in tall. The focal length is 13 in. So the horizontal FOV is around 30.14 degrees, while the vertical FOV is 42.07 degrees.



Figure 4: Two photos of pinhole camera

2.2 Use your pinhole camera

The pinhole diameters used were 0.7mm, 2mm, and 4mm. In general, I found that the larger pinholes produced blurrier results, but the smaller pinholes produced much dimmer projections, requiring a significantly slower shutter speed to capture a visible image. In addition,

images taken using the 0.7mm pinhole contained a ring-like artifact. This is probably due to the fact that the hole used to insert the camera was not light-tight, and this became noticeable as less light entered through the pinhole.



Figure 5: Three scenes captures with the 4mm pinhole (left), the 2mm pinhole (center), and the 0.7mm pinhole (right)

2.3 Bonus: Camera obscura in your room



Figure 6: Left: Pinhole over window. Right: Projection on window view